

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Redes

Semestre II – 2026



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

Laboratorio 1

Esquemas de comunicación e introducción a Wireshark

Reporte individual

Integrante:

Nájera Marakovits, Ingrid Nina Alessandra – 231088

Guatemala, 14 de julio de 2026

1. Introducción

En este laboratorio se trabajaron diferentes formas de transmitir información y se utilizó Wireshark para observar cómo se comunican los dispositivos por medio de paquetes. La primera parte permitió comparar los códigos Morse y Baudot, además de comprender las dificultades que aparecen al transmitir mensajes y utilizar un conmutador.

En la segunda parte se personalizó el entorno de Wireshark, se identificó la interfaz de red utilizada por la computadora, se configuró una captura con ring buffer y se analizaron paquetes relacionados con el protocolo HTTP. Esta actividad permitió observar de una forma más clara información como las direcciones de origen y destino, los protocolos utilizados, las banderas TCP y el contenido de las solicitudes y respuestas HTTP.

2. Primera parte: Esquemas de comunicación

Durante la transmisión de mensajes se utilizaron los códigos Morse y Baudot. El código Morse fue el más fácil de comprender y transmitir, debido a que se basa en pulsos cortos y largos que se pueden distinguir con facilidad. Sin embargo, en Baudot ocurrieron menos errores después de practicarlo, ya que cada letra siempre se representa mediante una secuencia de cinco bits.

La transmisión mediante notas de voz permitió volver a escuchar los mensajes cuando alguna parte no se entendía. A pesar de esto, también fue más difícil corregir un error inmediatamente, porque primero era necesario escuchar el mensaje completo. Además, una pausa incorrecta podía confundirse con la separación entre letras o palabras.

La introducción de un conmutador permitió centralizar la comunicación y reenviar cada mensaje a su destino correspondiente. Esto facilita la organización, pero también puede ocasionar retrasos si el conmutador recibe demasiados mensajes al mismo tiempo. Agregar más conmutadores puede distribuir la carga, aunque también aumenta la complejidad del sistema.

3. Segunda parte: Introducción a Wireshark

3.1. Personalización del entorno

Primero se creó un perfil llamado Nina Nájera. Esto permitió guardar las configuraciones del laboratorio sin modificar directamente el perfil predeterminado de Wireshark.

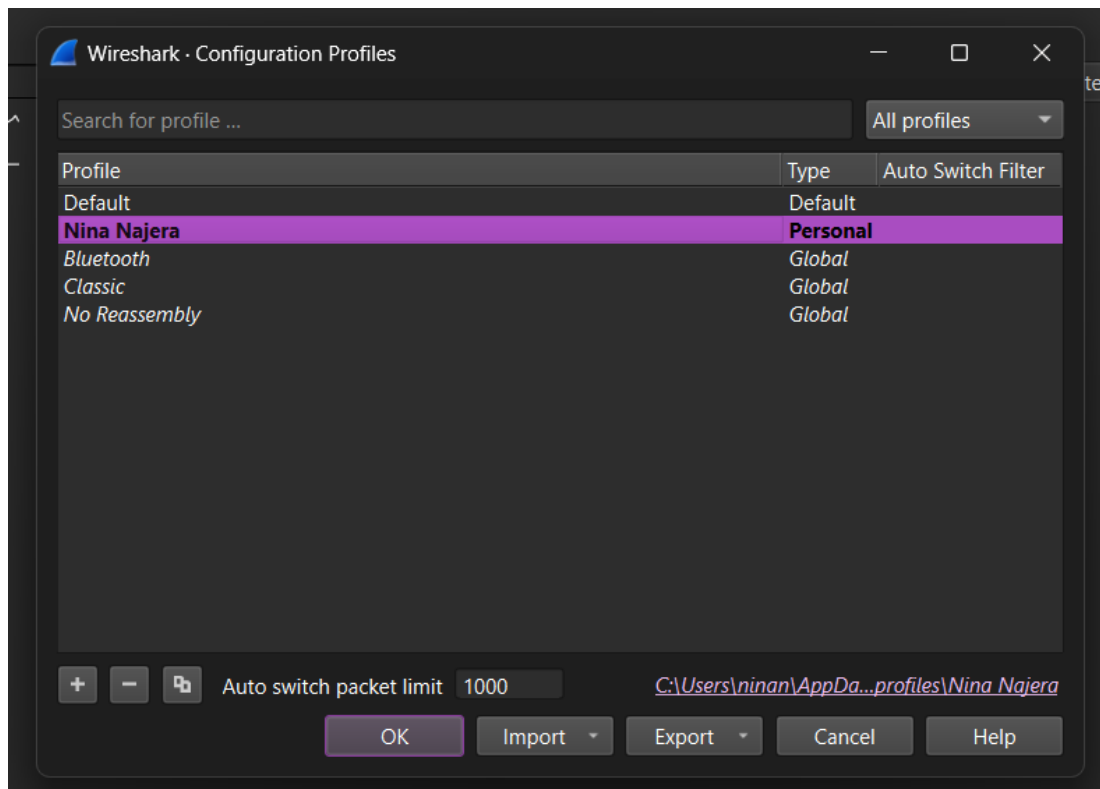


Figura 1: Perfil personal creado en Wireshark

También se modificó la distribución de los paneles, se cambió el formato de la hora y se agregó una columna para mostrar la longitud de los paquetes. La columna de longitud original fue ocultada para evitar mostrar información repetida.

Se creó una regla de color para los paquetes TCP cuya bandera SYN fuera igual a uno. El filtro utilizado fue:

```
tcp.flags.syn == 1
```

Además, se creó un botón llamado *TCP SYN* para aplicar el filtro de manera rápida.

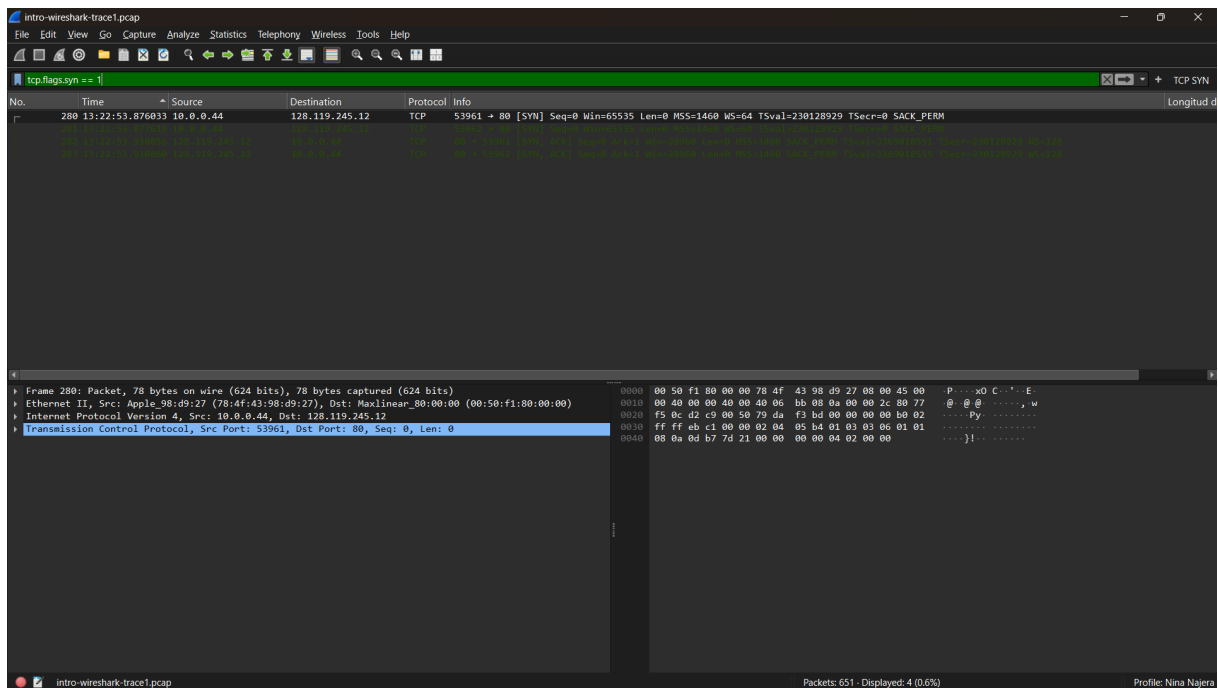


Figura 2: Aplicación del botón y filtro TCP SYN

En la captura se observan solamente los paquetes que cumplen con el filtro. También puede verse el perfil activo, la columna personalizada y la nueva distribución de los paneles.

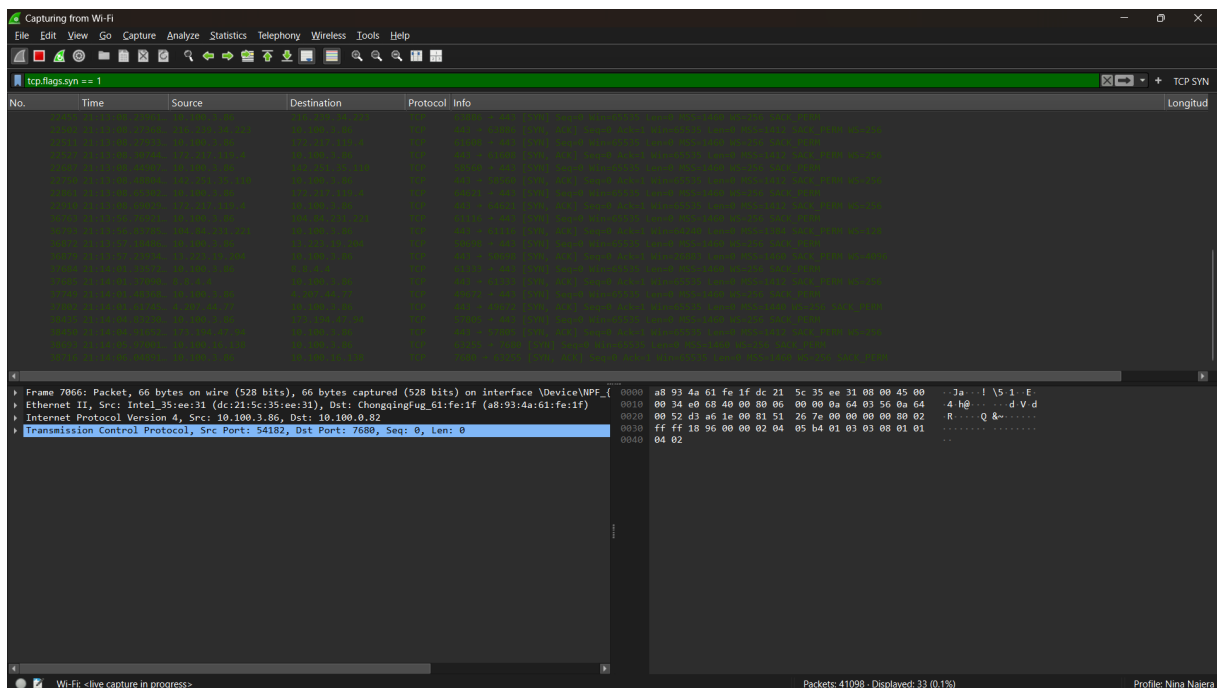


Figura 3: Captura de paquetes utilizando el perfil personalizado

3.2. Selección de la interfaz

En las opciones de captura se seleccionó únicamente la interfaz Wi-Fi, debido a que era la interfaz utilizada para conectarse a la red. Las interfaces de Bluetooth, loopback, Ethernet y conexiones virtuales no fueron seleccionadas.

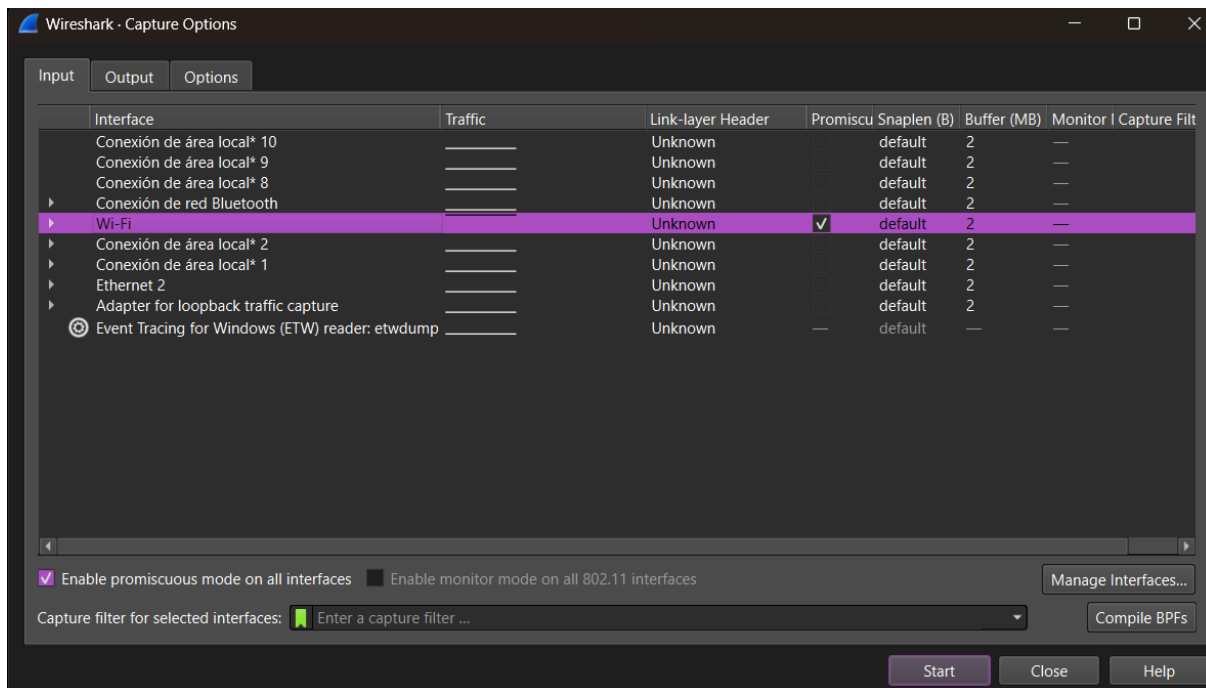


Figura 4: Selección de la interfaz Wi-Fi

3.3. Configuración de red

Se ejecutó el comando:

```
ipconfig
```

Este comando muestra la configuración TCP/IP de las interfaces instaladas en Windows. En el adaptador Wi-Fi se observaron los siguientes valores:

- Dirección IPv4: 10.100.3.86
- Máscara de subred: 255.255.224.0
- Puerta de enlace predeterminada: 10.100.0.1

La dirección IPv4 identifica al equipo dentro de la red local. La máscara de subred permite determinar qué parte de la dirección representa la red y qué parte identifica al dispositivo. La puerta de enlace corresponde al dispositivo utilizado para comunicarse con otras redes, como Internet.

También se observó una dirección IPv6 local y adaptadores desconectados, como la conexión de red Bluetooth.

```
Sufijo DNS específico para la conexión. . . :
Adaptador de LAN inalámbrica Wi-Fi:
Sufijo DNS específico para la conexión. . . :
Vínculo: dirección IPv6 local. . . : fe80::e6d2:b795:97a1:a015%18
Dirección IPv4. . . . . : 10.100.3.86
Máscara de subred. . . . . : 255.255.224.0
Puerta de enlace predeterminada. . . . . : 10.100.0.1
Adaptador de Ethernet Conexión de red Bluetooth:
Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . . :
C:\Users\ninan>
```

Figura 5: Resultado del comando ipconfig

3.4. Configuración del ring buffer

Para la captura de paquetes se configuró un ring buffer con archivos de cinco megabytes y un máximo de diez archivos. Se utilizó el nombre base:

lab1_231088.pcap

La creación de un nuevo archivo se configuró para ocurrir cada vez que la captura alcanzara cinco megabytes. Al llegar al máximo de diez archivos, Wireshark puede reemplazar los archivos más antiguos, evitando que la captura continúe utilizando espacio de almacenamiento sin límite.

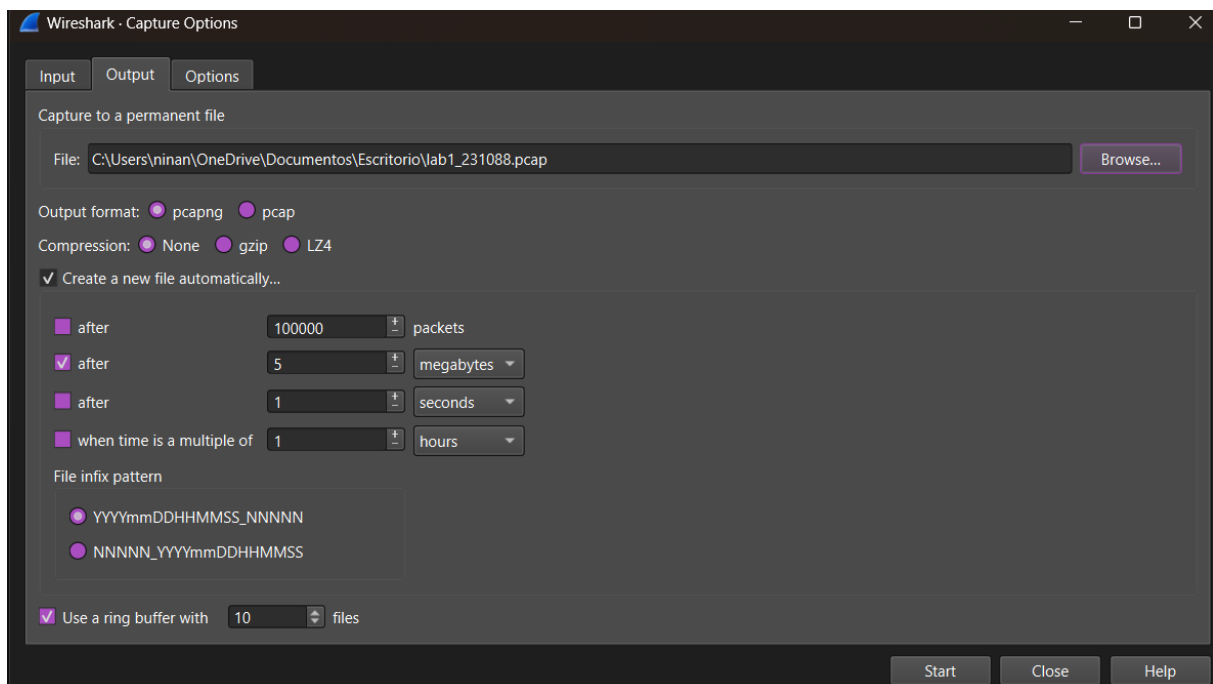


Figura 6: Configuración del ring buffer

Para generar tráfico se navegaron diferentes sitios y se mantuvo la captura activa hasta que se crearon los archivos. Como resultado, se generaron diez archivos. La mayoría alcanzó aproximadamente cinco megabytes y el último archivo tuvo un tamaño menor porque la captura fue detenida antes de que se llenara.

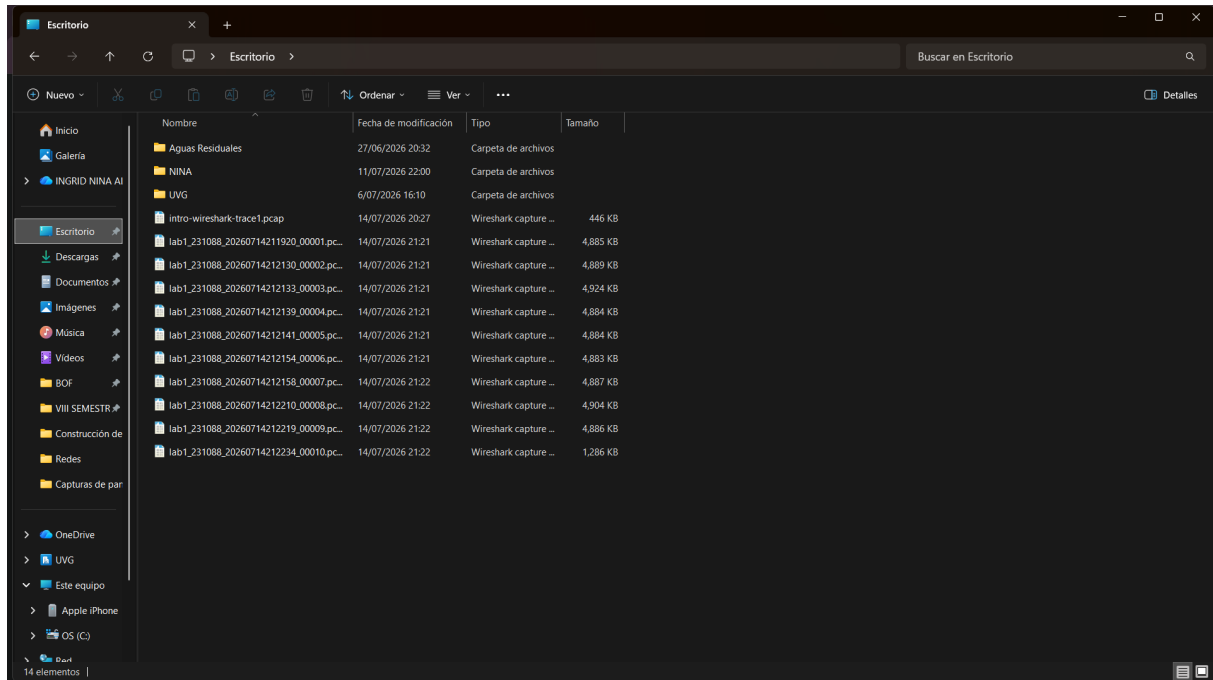


Figura 7: Archivos generados mediante el ring buffer

4. Análisis de paquetes HTTP

Para analizar el protocolo HTTP se identificó la solicitud realizada hacia el archivo:

`INTRO-wireshark-file1.html`

En la lista de paquetes se observó una solicitud GET desde la dirección del cliente hacia el servidor. Después se observó una respuesta con el estado **200 OK**, lo cual indica que la solicitud fue procesada correctamente y que el contenido fue devuelto.

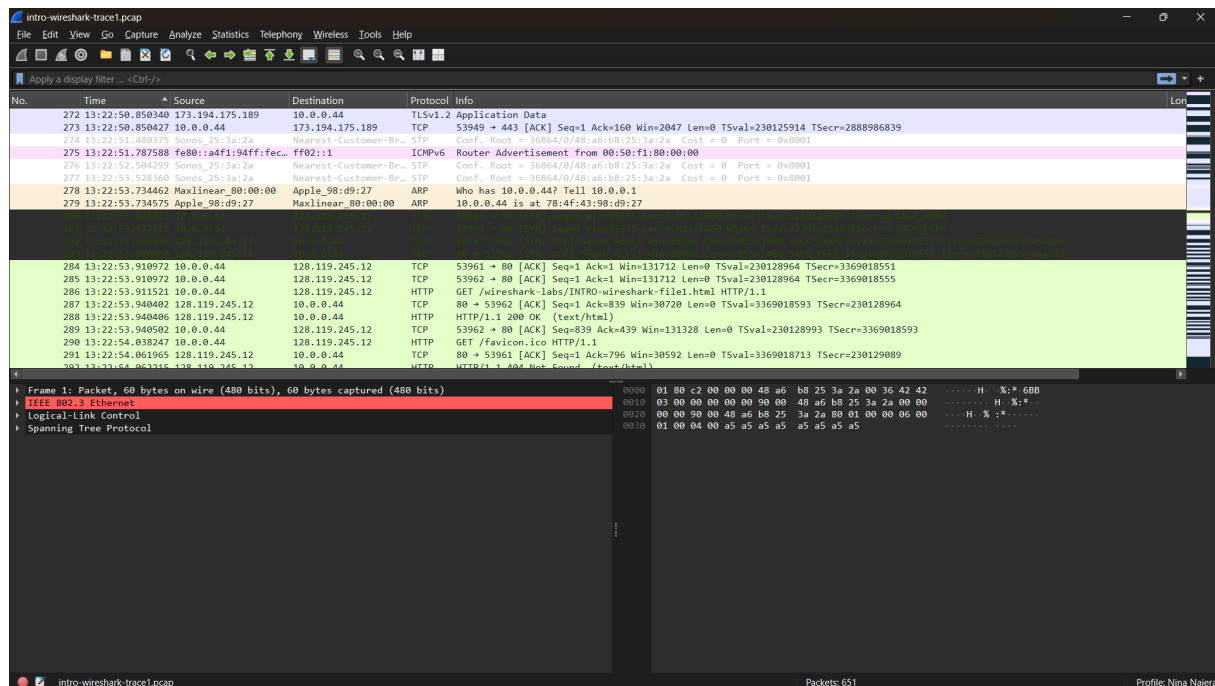


Figura 8: Solicitud GET y respuesta HTTP 200 OK

4.1. Resultados

- a. ¿Qué versión de HTTP está ejecutando el navegador?

La solicitud enviada por el navegador muestra:

GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1

Por lo tanto, el navegador está utilizando HTTP/1.1.

- b. ¿Qué versión de HTTP está ejecutando el servidor?

La respuesta recibida contiene:

HTTP/1.1 200 OK

Por lo tanto, el servidor también está utilizando HTTP/1.1.

- c. ¿Qué lenguajes indica el navegador que acepta?

En el encabezado Accept-Language se indicó el valor:

en-US, en;q=0.5

Esto indica que el navegador acepta principalmente inglés de Estados Unidos y también inglés general.

- d. ¿Cuántos bytes de contenido fueron devueltos por el servidor?

En el campo Content-Length de la respuesta se indicó un contenido de 128 bytes. Este valor representa el tamaño del cuerpo del recurso enviado por el servidor y no necesariamente el tamaño total del paquete, ya que el paquete también contiene encabezados de los distintos protocolos.

- e. ¿Dónde convendría escuchar los paquetes ante un problema de rendimiento?

Sería conveniente realizar capturas cerca del cliente y cerca del servidor para comparar el tiempo que tardan las solicitudes y respuestas en ambos extremos. También se podrían observar los paquetes en un router o switch intermedio mediante un puerto configurado para copiar tráfico.

Instalar Wireshark directamente en el servidor puede ser útil para una revisión puntual y autorizada, especialmente cuando se sospecha que el retraso ocurre en el servidor. Sin embargo, no siempre es la primera opción, ya que una captura grande puede consumir recursos y almacenar información sensible. Una alternativa es realizar la captura durante un tiempo limitado y analizar posteriormente el archivo desde otro equipo.

5. Discusión

La primera parte del laboratorio permitió notar que una comunicación que parece sencilla puede presentar errores por el ritmo, las pausas, el ruido o la interpretación de quien recibe el mensaje. También se observó que cada esquema tiene ventajas diferentes. Morse fue más sencillo de utilizar, mientras que Baudot ofreció una estructura más uniforme.

La parte de Wireshark permitió observar de manera real la información que viaja por la red. Al principio, la cantidad de paquetes y protocolos puede ser difícil de interpretar, pero los filtros, las reglas de color y las columnas personalizadas ayudan a encontrar la información necesaria.

La configuración del ring buffer fue útil para comprender cómo se pueden realizar capturas largas sin crear un solo archivo demasiado grande. También fue interesante observar que acciones normales, como abrir una página o reproducir contenido, generan una gran cantidad de paquetes en pocos minutos.

Finalmente, el análisis HTTP permitió relacionar lo observado en Wireshark con el funcionamiento de una página web. Se identificó la solicitud del navegador, la respuesta del servidor y algunos encabezados utilizados para intercambiar información.

6. Conclusiones

1. Los códigos Morse y Baudot permiten representar información de distintas formas, pero ambos requieren reglas claras para evitar errores durante la transmisión.

2. Wireshark permite capturar y analizar paquetes con información sobre direcciones, protocolos, puertos, banderas y contenido intercambiado entre dispositivos.
3. Los filtros y las reglas de color facilitan el análisis cuando una captura contiene una gran cantidad de paquetes.
4. El ring buffer permite limitar el tamaño y la cantidad de archivos generados durante una captura extensa.
5. El análisis HTTP permitió identificar la solicitud realizada por el navegador y la respuesta entregada por el servidor mediante HTTP/1.1.